

Algoritmi e Strutture Dati – Parte B – 03/02/25

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Numeri irripetibili – Punti ≥ 8

Scrivere un algoritmo

```
printNumbers(int n)
```

che stampa tutti i numeri compresi fra 0 (incluso) e 10^n (escluso) in cui nessuna cifra sia ripetuta.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, se $n = 2$, il programma deve stampare tutti i numeri compresi fra 0 e 99, ad esclusione dei numeri 11, 22, 33, 44, 55, 66, 77, 88, 99. Se $n = 3$, non può stampare numeri come 909, 112, etc.

B2 – Giardinaggio – Punti ≥ 10

Sia P un vettore che descrive un insieme di n piante, dove $P[i]$ è il volume minimo di un vaso affinché la pianta i possa sopravvivere; sia V un vettore che descrive un insieme di m vasi, dove $V[j]$ è la dimensione del vaso j . In altre parole, la pianta $P[i]$ può essere messa nel vaso j se $P[i] \leq V[j]$. Ogni vaso può contenere al massimo una pianta.

Scrivere un algoritmo

```
int garden(int[] P, int n, int[] V, int m)
```

che prenda in input i vettori P e V contenenti n e m interi positivi e restituisca il numero massimo di piante che possono essere invase rispettando il vincolo della dimensione.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, sia $P = [3, 4, 2]$ e $V = [3, 3, 1]$; è possibile utilizzare i due vasi da 3 e invadere le piante che richiedono dimensione 3 e 2; non è possibile invadere la pianta che richiede 4. In questo caso l'algoritmo dovrà restituire 2.

B3 – Mosse matriciali – Punti ≥ 12

Si consideri una matrice A di dimensione $n \times n$ contenente numeri interi compresi tra 1 e n . Ogni elemento della matrice rappresenta il numero massimo di passi che una pedina può fare partendo da quella casella, in verticale verso il basso o in orizzontale verso destra, ma non in diagonale.

Quando una pedina si trova nella casella (r, c) , può fare una *mossa* spostandosi in basso o a destra di un numero di passi inferiore o uguale al valore contenuto nella casella stessa. In altre parole, la pedina si può spostare in una qualunque delle caselle $(r + \Delta, c)$ o $(r, c + \Delta)$, con $1 \leq \Delta \leq A[r][c]$, purché il movimento non esca dai limiti della matrice.

Scrivere un algoritmo

```
int minSteps(int[][] A, int n)
```

che prenda in input la matrice A e restituisca il *numero minimo di mosse* necessarie per andare dalla posizione $(1, 1)$ (l'angolo in alto a sinistra) alla posizione (n, n) (l'angolo in basso a destra).

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, nella matrice seguente è evidenziato un percorso minimo di 3 mosse, pertanto l'algoritmo dovrà restituire 3 in questo caso.

3	1	3	1
1	1	1	1
1	2	2	2
1	1	3	1

